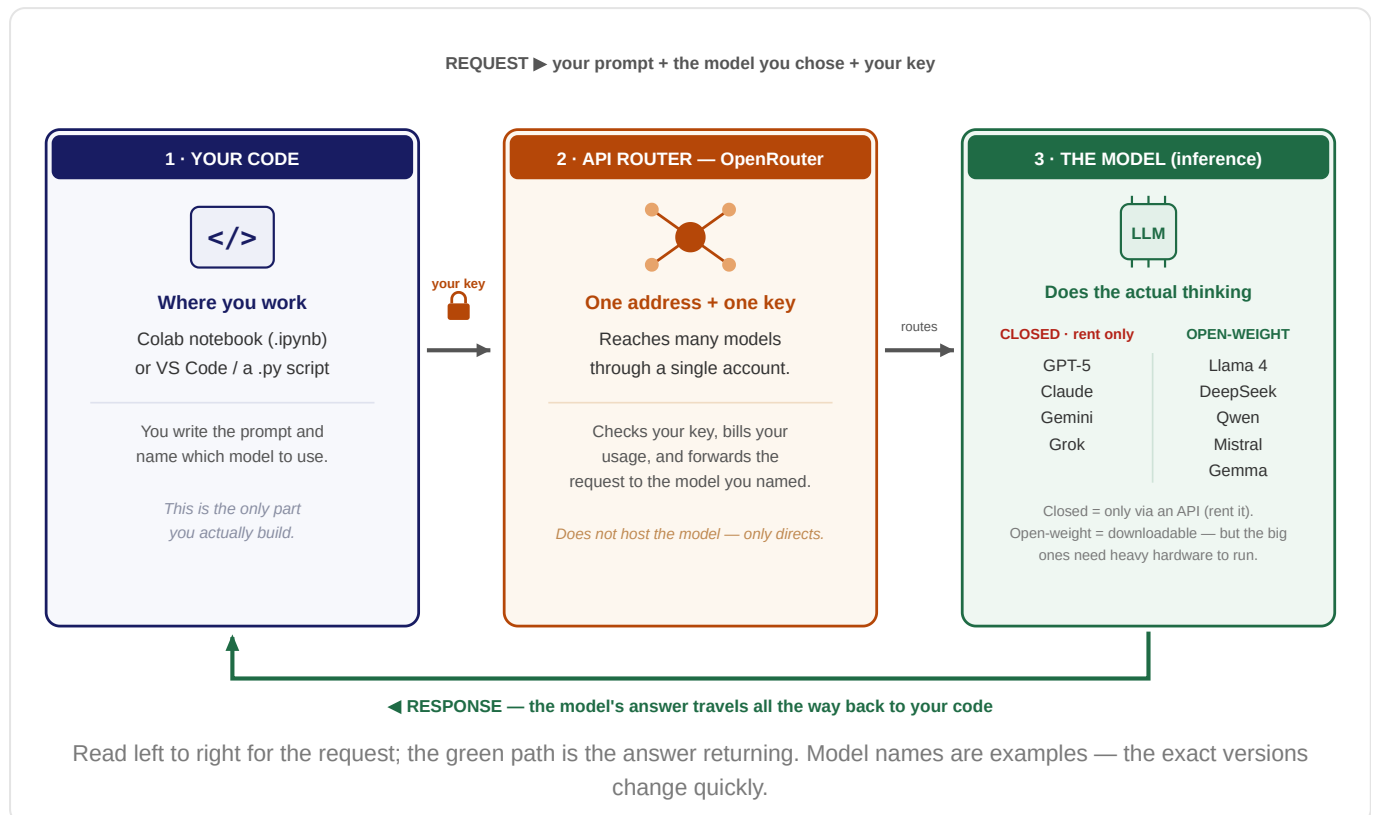


Your code, OpenRouter, and the model

Three different things — and how they connect so your code (or notebook) can use a large language model.

Several of you told us the words blur together: your **script**, the **OpenRouter key**, and "**the model**." They are three separate things that sit in a line. Your code asks a question; OpenRouter is the counter that takes the order and checks who you are; the model is what actually thinks and answers. Read the diagram left to right — a request travels right, and the answer comes back.



IMPORTANT — OPENROUTER IS ONLY THE ROUTER, NOT WHERE THE MODEL LIVES

OpenRouter does not run any model itself. The model actually runs on the computers of an **inference provider** — a company with racks of specialised chips (GPUs) in a data centre. OpenRouter simply forwards your request to one of them and relays the answer back.

For **closed** models, only the maker — or a cloud partner it has authorised — can serve them, because the weights are secret: OpenAI's GPT-5 (also offered on Microsoft Azure), Anthropic's Claude (also on Amazon Bedrock and Google Vertex AI), Google's Gemini. No independent company can host these. For **open-weight** models, any capable inference company can host them — for example **Together AI, Fireworks, DeepInfra, or Groq** — often several serving the same model, and OpenRouter picks one for you.

The point stands either way: a closed model only ever comes from its maker's official channel; an open-weight model can come from many competing hosts.

A PLAIN ANALOGY — THE FOOD COURT

Think of a food court. **You** (your code) decide what you want and write the order. **OpenRouter** is the single ordering counter: you show your membership card (your key) so they know who to bill, you say which stall you want, and they pass your order to that kitchen. **The kitchen** (the model) is what actually cooks. And the kitchens are not part of the counter — they are run by different catering companies (the inference providers), often in another building; the counter only relays your order to the right one. You never talk to a kitchen directly — one counter, one card, and you can order from any stall. That is the whole point of a router.

The three, side by side

	What it is	What you provide	Examples
1 · Your code	The script or notebook you write and run. It sends the request and displays the answer.	The prompt, and the name of the model you want.	A Colab <code>.ipynb</code> ; a <code>.py</code> file in VS Code.
2 · OpenRouter (the API router)	A middleman service. One account and one key let you reach many models. It authenticates you, bills you, and forwards the request.	Nothing to build — you just send your key with each request.	OpenRouter. (Others exist: Together, Fireworks, or a provider's own API.)
3 · The model (inference)	The large language model that actually reads your prompt and generates the reply. "Inference" just means the model running to produce an answer.	Nothing — it runs on the provider's computers.	Closed: GPT-5, Claude, Gemini, Grok. Open-weight: Llama 4, DeepSeek, Qwen, Mistral, Gemma.

Where each word you have heard fits

- **"OpenRouter key"** — your password to the **router (2)**. It proves who you are and lets them bill your usage. It is **not** the model, and it is not your code.
- **"Script" / "my code" / "the notebook"** — **layer 1**. The part you write. It holds your key and the model name, and sends the request.
- **"Model serving via OpenRouter"** — the round trip: OpenRouter hands your request to the **model (3)** — which runs on an inference company's computers — the model produces an answer, and it comes back to your code. You choose the model by its name, e.g. `openai/gpt-4o-mini`.

You already did this in Class 1

Both live demos in Class 1 were this exact diagram in action — your notebook, through OpenRouter, to a model and back. Nothing new; you have run it already.

Class 1 notebook	What the call did	What it showed you
Classifier vs API (Capsule 2)	Sent each product review to a model through OpenRouter and asked for one word back — positive or negative — with zero training examples.	A foundation model can do the task with no labelled data. Switching the question to "does this review mention delivery?" was just rewriting the prompt — same key, same router, same model.
Tokenizer demo (Capsule 3)	The optional live cell asked a model, through OpenRouter, "how many r's are in <i>strawberry</i> ?"	Let you watch a real model answer (and sometimes get it wrong) — the same call pattern, pointed at a different question.

In both notebooks, the only things that changed between the free "off" mode and the live call were the **RUN_LIVE** switch and your key. The router address and the shape of the call stayed identical.

The same idea, in the code you already run

Every live cell in the Class notebooks is really just these four pieces. The colours match the three layers above.

```
# from your notebook — one call, three layers in it
url      = "https://openrouter.ai/api/v1/chat/completions" # 2 · the router's address
headers = { "Authorization": "Bearer sk-or-v1-...your key..." } # 2 · your key, checked here
body     = { "model": "openai/gpt-4o-mini",                  # 3 · which model runs
             "messages": [{"role": "user", "content": PROMPT}] } # 1 · your request
answer = send(url, headers, body) # 1 · the reply comes back to your code
```

One line to remember: **your code** names the model and carries the key → **OpenRouter** checks the key and routes → **the model** runs and answers → the reply returns to **your code**. Change the model name and you switch kitchens without changing anything else.

PE6201 · Emerging AI Technologies · MSc Enterprise AI · Nanyang Technological University. Model names are current examples only and change frequently. There is no such thing as a question too basic — bring it to class.